

FUNDAMENTALS OF PROGRAMMING

Lecture 7: NESTED LOOPS

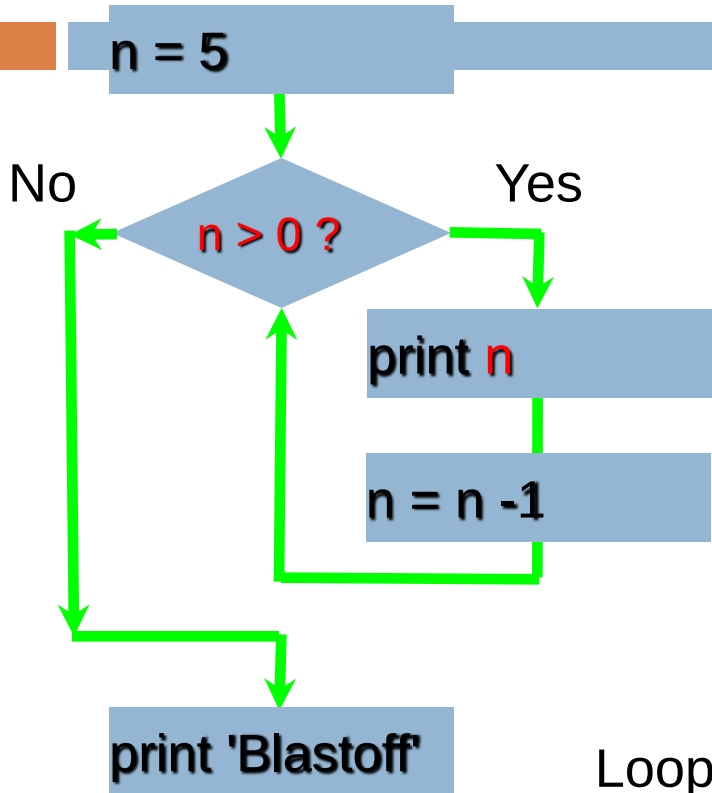
Instructor: Dr Aqib Perwaiz

FUNDAMENTALS OF PROGRAMMING CS-114

Lecture 7

LOOPS RECAP





Repeated Steps

Program:

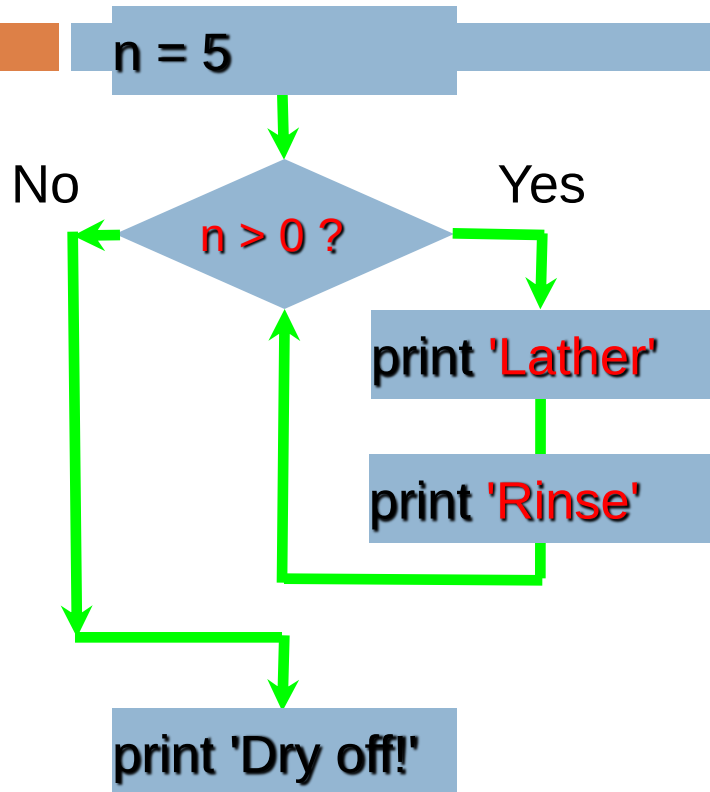
```
n = 5
while n > 0 :
    print n
    n = n - 1
print 'Blastoff!'
print n
```

Output:

5
4
3
2
1
Blastoff!
0

Loops (repeated steps) have **iteration variables** that change each time through a loop. Often these **iteration variables** go through a sequence of numbers.

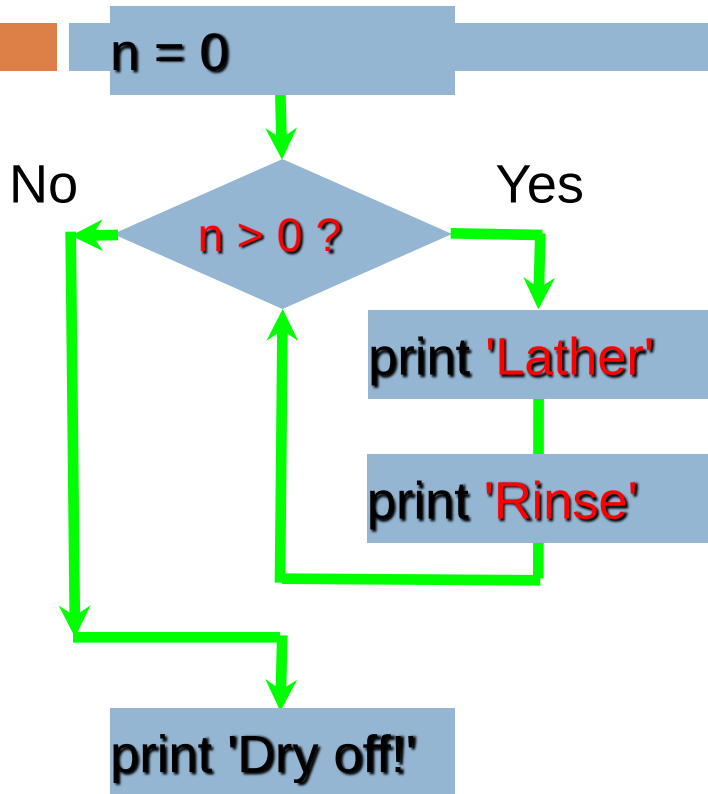
An Infinite Loop



```
n = 5
while n > 0 :
    print 'Lather'
    print 'Rinse'
print 'Dry off!'
```

What is wrong with this loop?

Another Loop



```
n = 0
while n > 0 :
    print 'Lather'
    print 'Rinse'
    print 'Dry off!'
```

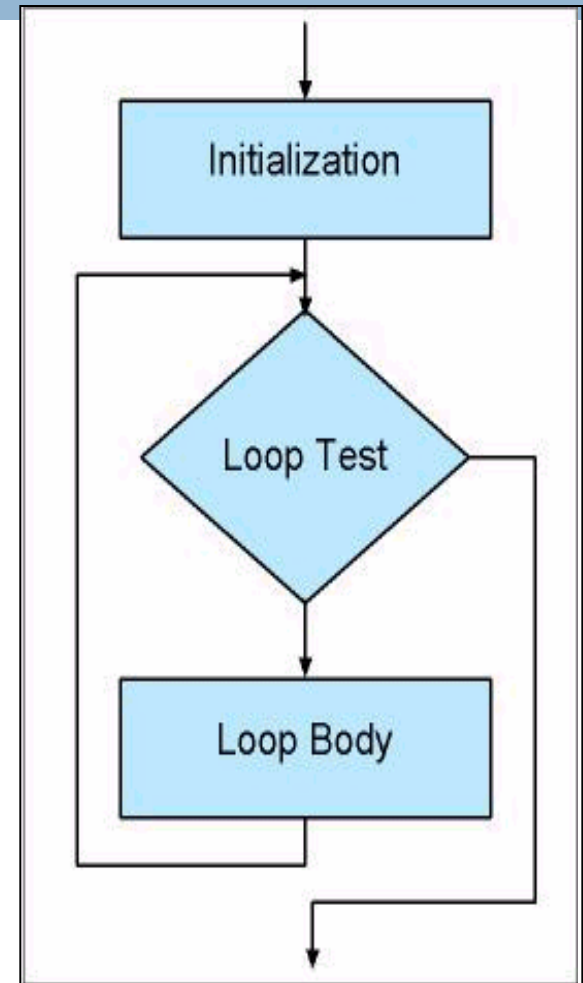
What does this loop do?

while Loop

- Normally loop does something a fixed number of times.
- What happens if you don't know how many times you want to do something before you start the loop?
- In this case a different kind of loop may be used: the while loop.

while Loop

- **while loop** executes a set of statements as long as the condition specified at the beginning is **true**.
- The condition is evaluated at the beginning of the loop
 - If it is true, the loop will execute
 - If it is false, the loop will not execute even once



While Loop Operation

- First, the boolean expression is evaluated
 - If false, the program skips to the line following the while loop
 - If true, the body of the loop is executed
 - During execution, some item from the boolean expression is changed
 - After executing the loop body, the boolean expression is checked again repeating the process until the expression becomes false
- A while loop might not execute at all if the boolean expression is false on the first check

Syntax of the *while* Statement

A Loop Body with Several Statements:

```
while (Boolean_Expression )  
{  
    Statement_1  
    Statement_2  
    ...  
    Statement_Last  
}
```

Do NOT put a semicolon here.

body

A Loop Body with a Single Statement:

```
while (Boolean_Expression )  
    Statement
```

body

while Loop

Syntax:

```
while(condition)
    {statement 1;
      ...
      statement n;
    }
```

Example

```
#include <iostream>
using namespace std;
int main()
{
    int n = 99; // make sure n isn't initialized to 0
    while( n != 0 ) // loop until n is 0
    {
        cin >> n; // read a number into n
    }
    cout << endl;
    return 0;
}
```

Sample output

- Here's some sample output. The user enters numbers, and the loop continues until 0 is entered, at which point the loop and the program terminate.

1

27

33

144

9

0

do-while

- Similar to while loop, except the condition is at the end of the loop.
 - The loop always executes at least once, regardless of whether the condition is true or not.
- **Syntax:**

```
do
    {statement 1;
    ...
    statement n; } while(<condition>);
```

Note: A semicolon at the end and curly braces are always required.

do-while

Output:

1
2
3

- **Example**

```
int a=1;  
do {  
    cout<<a<<endl;a++; }while(a<=3);
```

- However if,

```
int a=4;  
do {  
    cout<<a<<endl;a++; }while(a<=3);
```

Output:

4

do-while loop

- A variation of the while loop.
- A do-while loop is always executed at least once
 - The body of the loop is first executed
 - The boolean expression is checked after the body has been executed
- Syntax:
 - ```
do
{
 statements to repeat
}
while (boolean_expression);
```

## Syntax of the *do-while* Statement

---

### A Loop Body with Several Statements:

```
do
{
 Statement_1
 Statement_2
 ...
 Statement_Last
} while (Boolean_Expression);
```

The diagram illustrates the loop body for multiple statements. A large right-facing curly brace groups the statements *Statement\_1*, *Statement\_2*, *...*, and *Statement\_Last*. An arrow points from the center of this brace to the word *body*, indicating that these statements collectively form the loop body.

Do not forget the  
final semicolon.

### A Loop Body with a Single Statement:

```
do
 Statement
while (Boolean_Expression);
```

The diagram illustrates the loop body for a single statement. An arrow points from the word *Statement* to the word *body*, indicating that this single statement forms the loop body. Another arrow points from the semicolon at the end of the *while* line to the note 'Do not forget the final semicolon.'

# Infinite Loops

- Loops that never stop are infinite loops
- The loop body should contain a line that will eventually cause the boolean expression to become false
- Example: Print the odd numbers less than 12

```
x = 1;
while (x != 12)
{
 cout << x << endl;
 x = x + 2;
}
```

- Better to use this comparison: `while ( x < 12)`

# The for-Statement

- A for-Statement (for-loop) is another loop mechanism in C++
  - Designed for common tasks such as adding numbers in a given range
  - Is sometimes more convenient to use than a while loop
  - Does not do anything a while loop cannot do

# for/while Loop Comparison

- ```
int sum = 0;
  int n = 1;
  while(n <= 10) // add the numbers 1 - 10
  {
    sum = sum + n;
    n++;
  }
```
- ```
int sum = 0;
 int n = 0;
 for (n = 1; n <= 10; n++) //add the numbers 1 - 10
 sum = sum + n;
```

# For Loop Dissection

- The for loop uses the same components as the while loop in a more compact form

- `for (n = 1; n <= 10; n++)`

# for Loop Alternative

- A for loop can also include a variable declaration in the initialization action
  - **`for (int n = 1; n <= 10; n++)`**
  - This line means
    - Create a variable, n, of type int and initialize it with 1
    - Continue to iterate the body as long as  $n \leq 10$
    - Increment n by one after each iteration

# The for-loop Body

- The body of a for-loop can be
  - A single statement
  - A compound statement enclosed in braces
    - Example:

```
for(int number = 1; number >= 0; number--)
{
 // loop body statements
}
```

## The *for* Statement

### *for* Statement

#### Syntax

```
for (Initialization_Action; Boolean_Expression; Update_Action)
 Body_Statement
```

#### Example

```
for (number = 100; number >= 0; number--)
 cout << number
 << " bottles of beer on the shelf.\n";
```

### Equivalent *while* loop

#### Equivalent Syntax

```
Initialization_Action;
while (Boolean_Expression)
{
 Body_Statement
 Update_Action;
}
```

#### Equivalent Example

```
number = 100;
while (number >= 0)
{
 cout << number
 << " bottles of beer on the shelf.\n";
 number--;
}
```

#### Output

```
100 bottles of beer on the shelf.
99 bottles of beer on the shelf.
.
.
.
0 bottles of beer on the shelf.
```



# Break Statement Revisited

26

- **break statement can be used with Switch or any of the 3 looping structures**
- **it causes an immediate exit from the Switch, while, do-while, or for statement in which it appears**
- **if the break is inside nested structures, control exits only the innermost structure containing it**

# Continue Statement

27

- is valid only within loops
- terminates the current loop iteration, but not the entire loop
- in a for or while, continue causes the rest of the body statement to be skipped--in a for statement, the update is done
- in a do-while, the exit condition is tested, and if true, the next loop iteration is begun

# The break Statement

```
int j = 40;
while (j < 80){
 j += 10;
 if (j == 70)
 break;
 cout << "j is " << j << '\n' ;
}
cout << "We are out of the loop as j=70.\n";
//see useBreak1.cpp
```

j is 50

j is 60

We are out of the loop as j=70.

# The continue Statement

```
int j = 40;
while (j < 80) {
 j += 10;
 if (j == 70)
 continue; //skips the 70
 cout << "j is " << j << "\n";
} //see UseContinue1.cpp
cout << "We are out of the loop" << endl;
```

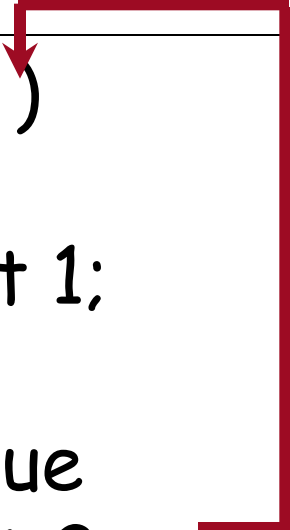
j is 50

j is 60

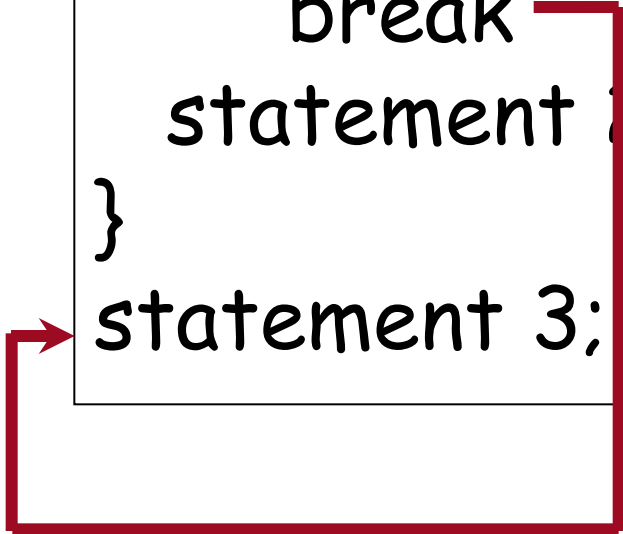
j is 80

We are out of the loop.

# break and continue



```
while (- - -)
{
 statement 1;
 if(- - -)
 continue
 statement 2;
}
statement 3;
```



```
while (- - -)
{
 statement 1;
 if(- - -)
 break
 statement 2;
}
statement 3;
```

# Nested Control Structures

31

- Suppose we want to create the following pattern

```
*

* *

* * *

* * * *

* * * * *
```

- In the first line, we want to print one star, in the second line two stars and so on





# Nested Control Structures (continued)

34

- Since five lines are to be printed, we start with the following for statement

```
for (i = 1; i <= 5 ; i++)
```

- The value of `i` in the first iteration is 1, in the second iteration it is 2, and so on
- Can use the value of `i` as limit condition in another for loop nested within this loop to control the number of starts in a line

# Nested Control Structures (continued)

35

- The syntax is:

```
for (i = 1; i <= 5 ; i++)
{
 for (j = 1; j <= i; j++)
 cout << "*";
 cout << endl;
}
```

# Nested loops

- A loop can be nested inside of another loop. C++ allows at least 256 levels of nesting.

```
#include <iostream>
using namespace std;
int main()
{
 int size;
 cout << "How large a triangle do you want? ";
 cin >> size;
 for (int r = 1; r <= size; r++)
 {
 for (int c = 1; c <= r; c++)
 {
 cout << "*";
 }
 cout << endl;
 }
 return 0;
}
```

**The result when  
the entered  
number is 7:**

**\***

**\*\***

**\*\*\***

**\*\*\*\***

**\*\*\*\*\***

**\*\*\*\*\***

**\*\*\*\*\***

# Nested Control Structures

## (continued)

37

- What pattern does the code produce if we replace the first for statement with the following?

```
for (i = 5; i >= 1; i--)
```

- Answer:

```
* * * * *
```

```
* * * *
```

```
* * *
```

```
* *
```

```
*
```

# Which Loop To Use?

- Choose the type of loop late in the design process
  - First design the loop using pseudocode
  - Translate the pseudocode into C++
  - The translation generally makes the choice of an appropriate loop clear
  - While-loops are used for all other loops when there might be occasions when the loop should not run
  - Do-while loops are used for all other loops when the loop must always run at least once

# Choosing a for-loop

- for-loops are typically selected when doing numeric calculations, especially when using a variable changed by equal amounts each time the loop iterates

# Choosing a while-loop

- A while-loop is typically used
  - When a for-loop is not appropriate
  - When there are circumstances for which the loop body should not be executed at all

# Choosing a do-while Loop

- A do-while-loop is typically used
  - When a for-loop is not appropriate
  - When the loop body must be executed at least once



# Acknowledgements

1. Deitel and Deitel: C++ How to Program, 7th Edition, Prentice Hall Publications
2. Robert Lafore: Object-Oriented Programming in C++, Fourth Edition, December 2001, Sams Publishing .
3. A Structured Programming Approach Using C++ by Behrouz A. Forouzan
4. [www.cplusplus.com](http://www.cplusplus.com)